Layr Labs

Orbit Updates Review

Version: 2.0

December, 2024

Contents

Introduction 2
- Disclaimer . . . 2
- Document Structure . . . 2
- Overview . . . 2

Security Assessment Summary 3
- Scope . . . 3
- Approach . . . 3
- Coverage Limitations . . . 4
- Findings Summary . . . 4

Detailed Findings 5

Summary of Findings 6
- Incorrect Encoding Chosen When `Post4844Blobs` Is Enabled . . . 7
- Failover May Use Both AnyTrust & EIP-4844 If Both Are Enabled . . . 8
- `failOver` Is Not Set When Failover To EIP-4844 Occurs . . . 10
- Overlapping Slot Usage With Future Arbitrum Nitro Versions . . . 12
- Miscellaneous General Comments . . . 13

A Vulnerability Severity Classification 14

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Layr Labs components in scope. The review focused solely on the security aspects of the Solidity implementation of the contract, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the smart contract in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Layr Labs components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see Vulnerability Severity Classification), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Layr Labs components in scope.

Overview

EigenDA is a Data Availability (DA) protocol that can be used to temporarily store data. This review focuses on a fork of the layer 2 optimistic rollup, Abitrum Nitro. The fork adds the functionality to use EigenDA as a data availability layer.

Updates have been made to the sequencer to allow posting of EigenDA batches. The batches are validated onchain through an EigenDA certificate. Offchain validators can fetch the required batch data through a query to an EigenDA proxy using the sequenced certificates.

Additionally, changes have been made to the fraud proof, to handle proving of EigenDA batches. The required changes include modifications to the preimage opcode to include KZG point verification.

Finally, a failover system has been implemented. This allows either AnyTrust or Ethereum's Native DA to be used when EigenDA is unresponsive.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the Layr Labs forks of orbit-actions, nitro-contracts and nitro repositories.

The scope of this time-boxed review was strictly limited to the files at orbit-actions@d3a071e, nitro-contracts@af0733f and nitro@206560b. Focusing on the following Pull Requests:

- `orbit-actions`
  - #1
- `nitro`
  - #41
  - #46
- `nitro-contracts`
  - #32
  - #33
  - #34

The fixes of the identified issues were assessed at commits orbit-actions@d3a071e, nitro-contracts@85aa27d and nitro@bba6f61.

Note: third party libraries and dependencies, such as OpenZeppelin smart contracts, were excluded from the scope of this assessment.

Approach

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

For a more detailed, but non-exhaustive list of examined vectors, see [1, 2].

To support this review, the testing team also utilised the following automated testing tools:

- Mythril: `https://github.com/ConsenSys/mythril`
- Slither: `https://github.com/trailofbits/slither`
- Surya: `https://github.com/ConsenSys/surya`
- Aderyn: `https://github.com/Cyfrin/aderyn`

For the Golang libraries and modules, the review focused on internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Go runtime. The manual review explored known Golang antipatterns such as integer overflow, floating point underflow, deadlocking, race conditions, memory and CPU exhaustion attacks and a multitude of panics including but not limited to `nil` pointer deferences, index out of bounds and calls to `panic()`.

To support this review, the testing team also utilised the following automated testing tools:

- golangci-lint: https://golangci-lint.run/
- vet: https://pkg.go.dev/cmd/vet
- errcheck: https://github.com/kisielk/errcheck

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 5 issues during this assessment. Categorised by their severity:

- Medium: 1 issue.
- Low: 2 issues.
- Informational: 2 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Layr Labs components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: Vulnerability Severity Classification.

A number of additional properties of the contracts, including gas optimisations, are also described in this section and are labelled as "informational".

Each vulnerability is also assigned a **status**:

- ***Open:*** the issue has not been addressed by the project team.
- ***Resolved:*** the issue was acknowledged by the project team and updates to the affected contract(s) have been made to mitigate the related risk.
- ***Closed:*** the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
ODA-01	Incorrect Encoding Chosen When `Post4844Blobs` Is Enabled	Medium	Resolved
ODA-02	Failover May Use Both AnyTrust & EIP-4844 If Both Are Enabled	Low	Closed
ODA-03	`failOver` Is Not Set When Failover To EIP-4844 Occurs	Low	Resolved
ODA-04	Overlapping Slot Usage With Future Arbitrum Nitro Versions	Informational	Resolved
ODA-05	Miscellaneous General Comments	Informational	Resolved

ODA-01	Incorrect Encoding Chosen When `Post4844Blobs` Is Enabled		
Asset	`nitro/arbnode/batch_poster.go`		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

When `PostEigenDA` and `Post4844Blobs` are enabled and `use4844 = true`, the blob encoding will always use EIP-4844 encoding, regardless of whether EigenDA should be used.

The issue occurs in the function `maybePostSequencerBatch()`, when the variable(s) `use4844` is set to `true`. Noting there are additional conditions for `use4844` to be set as `true`, such as requiring the last header to supply gas details. If `useEigenDA` is also set to `true`, then the blob will be stored in EigenDA on line [**1477**].

However, the encoding function `encodeAddBatch()` will always use EIP-4844 encoding if `use4844 = true`.

```
func (b *BatchPoster) encodeAddBatch(
    seqNum *big.Int,
    prevMsgNum arbutil.MessageIndex,
    newMsgNum arbutil.MessageIndex,
    l2MessageData []byte,
    delayedMsg uint64,
    use4844 bool,
    useEigenDA bool,
    eigenDaBlobInfo *eigenda.EigenDABlobInfo,
) ([]byte, []kzg4844.Blob, error) {
    methodName := sequencerBatchPostMethodName
    if use4844 { // @audit is used by default if both use4844 and useEigenDA are true
        methodName = sequencerBatchPostWithBlobsMethodName
    } else if useEigenDA {
        methodName = sequencerBatchPostWithEigendaMethodName
    }
```

As a result, the blob is posted to both EigenDA and EIP-4844. The function call on the sequencer will be `addSequencerL2BatchFromBlobs()` and thus the EigenDA blobs are never used.

Recommendations

To resolve this issue, alter the value of `b.building.use4844` depending on whether EigenDA posting is successful, only using EIP-4844 if failover is required.

Additionally, consider preventing the configuration from having `PostEigenDA` and `Post4844Blobs` enabled unless `EnableEigenDAFailover` is also set to true.

Resolution

The issue has been resolved in PR #52. The mitigation will disable EIP-4844 blobs when dispersal occurs via EigenDA.

ODA-02	Failover May Use Both AnyTrust & EIP-4844 If Both Are Enabled		
Asset	`nitro/arbnode/batch_poster.go`		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

If both AnyTrust and EIP-4844 blobs are enabled, the batch post may use both protocols.

When failover to AnyTrust occurs, it does not set `b.building.use4844` to false. Therefore, both protocols may be applied. First, AnyTrust will trigger dispersal, then output of that dispersal will be inserted in to an EIP-4844 blob.

```
if b.dapWriter != nil && !eigenDADispersed {
        if !b.redisLock.AttemptLock(ctx) {
            return false, errAttemptLockFailed
        }

        gotNonce, gotMeta, err := b.dataPoster.GetNextNonceAndMeta(ctx)
        if err != nil {
            batchPosterDAFailureCounter.Inc(1)
            return false, err
        }
        if nonce != gotNonce || !bytes.Equal(batchPositionBytes, gotMeta) {
            batchPosterDAFailureCounter.Inc(1)
            return false, fmt.Errorf("%w: nonce changed from %d to %d while creating batch", storage.ErrStorageRace, nonce,
                ↪  gotNonce)
        }
        // #nosec G115
        sequencerMsg, err = b.dapWriter.Store(ctx, sequencerMsg, uint64(time.Now().Add(config.DASRetentionPeriod).Unix()),
            ↪  config.DisableDapFallbackStoreDataOnChain)
        if err != nil {
            batchPosterDAFailureCounter.Inc(1)
            return false, err
        }

        batchPosterDASuccessCounter.Inc(1)
        batchPosterDALastSuccessfulActionGauge.Update(time.Now().Unix())

                // @audit should set `b.building.use4844 = false`
    }
```

The impact is rated as medium as the validator decoding will fail to unwrap both protocols, causing a halt in validation.

Recommendations

To resolve the issue, when AnyTrust dispersal occurs, set `b.building.use4844 = false`.

Furthermore, consider preventing failover to use both AnyTrust and Ethereum's DA when loading the configuration in `node.go`.

Resolution

This issue has been closed as invalid.

A check presently exists on line [**1223**] such that `use4844` will not be enabled unless `b.dapWriter == nil`. Therefore, it is not possible for both EIP-4844 and AnyTrust to be used together.

ODA-03	`failOver` Is Not Set When Failover To EIP-4844 Occurs		
Asset	`Contract.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The variable `failOver` is not set to `true` when a failover occurs and the backup protocol is Ethereum's EIP-4844 data availability.

When a failover occurs to AnyTrust, the following variables are set `b.building.useEigenDA = false` and `failOver = true`. These variables are not set for the Ethereum DA failover case.

The result is that an error will always be returned on line [**1507**]. The impact is rated as low as `b.eigenDAFailoverToETHDA = true` and therefore the next call to `maybePostSequencerBatch()` will have `useEigenDA = false` and process the batch using the Ethereum DA failover.

```
    if err != nil && errors.Is(err, eigenda.SvcUnavailableErr) && b.config().EnableEigenDAFailover && b.dapWriter != nil { //
          ↪  Failover to anytrust commitee if enabled
        log.Error("EigenDA service is unavailable, failing over to any trust mode")
        b.building.useEigenDA = false
        failOver = true
    }

    if err != nil && errors.Is(err, eigenda.SvcUnavailableErr) && b.config().EnableEigenDAFailover && b.dapWriter == nil { //
          ↪  Failover to ETH DA if enabled
                    // ... comments snipped for brevity
        log.Error("EigenDA service is unavailable and anytrust is disabled, failing over to ETH DA")
        b.eigenDAFailoverToETHDA = true // @audit should probably also set failOver = true, useEigenDA = false?

        // // if the batch's size exceeds the native DA max size limit, we must re-encode the batch to accomodate the AnyTrust,
              ↪  calldata, and 4844 size limits
        if (len(sequencerMsg) > b.config().MaxSize && !b.building.use4844) || (len(sequencerMsg) > b.config().Max4844BatchSize &&
              ↪  b.building.use4844) {
            batchPosterDAFailureCounter.Inc(1)
            batchPosterDAFailoverCount.Inc(1)

            b.building = nil
            return false, nil
        }

    }

    if err != nil && !failOver {
        batchPosterDAFailureCounter.Inc(1)
        return false, err // @audit always triggered for ETH DA case as `failOver` is not set

    } else if failOver {
        batchPosterDAFailoverCount.Inc(1)
    } else {
        batchPosterDASuccessCounter.Inc(1)
        batchPosterDALastSuccessfulActionGauge.Update(time.Now().Unix())
        eigenDADispersed = true
    }
}
```

Recommendations

It is recommended to set `b.building.useEigenDA = false` and `failOver = true` when failover to Ethereum DA occurs.

Resolution

The recommendation has been implemented in PR #50.

ODA-04	Overlapping Slot Usage With Future Arbitrum Nitro Versions
Asset	`nitro-contracts/src/bridge/SequencerInbox.sol`
Status	**Resolved:** See Resolution
Rating	Informational

Description

The current implementation of EigenDA's `SequencerInbox` will have overlapping slots with future version of Arbitrum's `SequencerInbox`.

The storage variable `eigenDARollupManager` will be slot 12 in EigenDA's `SequencerInbox`. Version `v2.1.0` of Arbitrum's `SequencerInbox` only uses 11 slots and therefore slot 12 is unused. However, v3.0.0-alpha-1 (the most recent version at the time of writing), has a value `buffer` which is stored at slot 12, overlapping with `eigenDARollupManager`. Any future upgrades from Arbitrum `v3.0.0` will cause complexities to resolve the unmatched slot numbering.

Recommendations

Consider adding a buffer to EigenDA's `SequencerInbox` of at least 50 empty slot values before the storage variable `eigenDARollupManager`. Thus, when Arbitrum adds new storage variable these may be added in place of the gaps to prevent overlapping slots.

To add gaps, a fixed sized array can be used, as seen in the following example.

```
uint256[50] __gap
```

Resolution

A gap of 38 slots has been added to `SequencerInbox` in PR #37.

ODA-05	Miscellaneous General Comments
Asset	All contracts
Status	**Resolved:** See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. **Fields Missing From Default Config**
   Related Asset(s): nitro/arbnode/batch_poster.go
   The configuration `EigenDABatchPosterConfig` does not include the field `EnableEigenDAFailover`.
   The issue is deemed of informational severity as it will default to `false`.
   It is recommended to explicitly add the field `EnableEigenDAFailover` to `EigenDABatchPosterConfig` and `TestBatchPosterConfig`.
   Similarly, add the following fields:
   - `DisableDapFallbackStoreDataOnChain`
   - `RedisLock`
   - `ReorgResistanceMargin`
2. **`failover` vs `failOver`**
   Related Asset(s): /*
   There is inconsistent usage of the word failover. Some occurrences consider it two words and capitalise 'O' and others do not.
   For example `failOver` defined on line 1461 on `batch_poster.go`. As opposed to `eigenDAFailoverToETHDA` defined on line 129.
   Treat failover as a single word and use lowercase for the 'o' in over to ensure consistency.
3. **Typos & Spelling Mistakes**
   Related Asset(s): `nitro/arbnode/batch_poster.go`
   - line 1488 has an extra *"could worst case"*.
   - line 1495 has double the comments slashes `// //`.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team have addressed the issues where applicable.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurance. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	Likelihood: Low	Likelihood: Medium	Likelihood: High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

References

[1] Sigma Prime. Solidity Security. Blog, 2018, Available: https://blog.sigmaprime.io/solidity-security.html. [Accessed 2018].

[2] NCC Group. DASP - Top 10. Website, 2018, Available: http://www.dasp.co/. [Accessed 2018].

σ'